DEVELOPMENT AND DEPLOYMENT OF AN EXPLAINABLE DEEP LEARNING SYSTEM FOR PNEUMONIA SCREENING

Technical Report on a Clinical Decision Support System (CDSS) utilizing MobileNetV2 Transfer Learning, Grad-CAM Saliency Mapping, and Automated LLM Reporting.

Submitted By:	Sathya S
GitHub Repository:	github.com/SathyaS24/AI-Medical-Intelligence-Platform
Live Application URL:	ai-medical-intelligence-platform-fjeb.onrender.com
Core Stack:	FastAPI, React, TypeScript, TensorFlow/Keras, SQLite, Docker, Render
Submission Date:	July 27, 2026

1. Executive Summary

Chest radiographs (CXRs) represent the primary imaging modality for diagnosing respiratory infections in pediatric settings. However, interpreting pediatric radiographs is complicated by anatomical variations, patient movement, and subjective differences between observers. This project reports on the technical implementation of MedTool AI, a clinical decision support system designed to assist radiologists by classifying frontal chest radiographs, highlighting focal zones of consolidation using Explainable AI (XAI) overlays, and generating structured draft clinical reports utilizing the Google Gemini API.

The system integrates a deep learning pipeline (MobileNetV2, fine-tuned on pediatric CXR scans) with a FastAPI backend server and an interactive web interface. To bridge the trust gap common in clinical AI tools, the system implements Gradient-weighted Class Activation Mapping (Grad-CAM), enabling clinicians to inspect the spatial features that influenced the model's diagnostic classification.

2. Dataset & Preprocessing Pipeline

The classification model was trained and evaluated on pediatric chest radiograph datasets containing anterior-posterior (AP) and posterior-anterior (PA) scans. Images are categorized into normal lungs (clear lung fields without active consolidations) and pneumonia (containing visible pulmonary consolidations, interstitial patterns, or pleural abnormalities).

To ensure robust evaluation and prevent data leakage, we implemented a strict 70:15:15 data partitioning strategy. The held-out testing partition is stored in a separate directory structure and is never exposed to the model during training or parameter optimization. A key challenge in multi-center radiograph analysis is the variation in exposure, contrast, and quality across imaging hardware. To address this, we integrated Contrast Limited Adaptive Histogram Equalization (CLAHE) into the preprocessing pipeline. CLAHE standardizes the contrast profiles of the lung fields, highlighting subtle consolidations while avoiding the amplification of background noise in soft tissues.

Input matrices are standardized as follows: image files are parsed, converted to 3-channel matrices, resized to 224x224 pixels, subjected to CLAHE contrast normalization, and normalized to floating-point values in the range [0.0, 1.0]. During local training, we apply online data augmentation—including random rotations, zooms, and horizontal flips—to improve the model's generalization.

3. Convolutional Neural Network Architecture

We selected the MobileNetV2 architecture as the core convolutional neural network. MobileNetV2 utilizes depthwise separable convolutions to dramatically reduce the number of parameters and computational complexity, making it ideal for clinical edge servers without dedicated GPUs. The base model is initialized with weights pre-trained on ImageNet. During the initial training phase, the convolutional base is frozen to retain general edge-detection features, and a customized classification head is appended to learn chest radiograph patterns:

```
Input Tensor (224x224x3) → MobileNetV2 Base → GlobalAveragePooling2D → Dense (128, ReLU) → Dropout (40%) → Dense (1, Sigmoid)
```

To address class distribution skew in the training set, we compute and apply inverse-frequency class weights during optimization, preventing the loss function from biasing toward the majority class. The final model is saved as a native Keras `.keras` checkpoint, accompanied by a metadata JSON file tracking its validation history, hyperparameters,

input parameters, and performance metrics (accuracy, loss, sensitivity). FastAPI loads this model in-memory during application startup, holding the weights cached to guarantee sub-second API response times.

4. Explainable AI (Grad-CAM) Implementation

Clinical adoption of deep learning tools depends heavily on transparency. To resolve the 'black-box' nature of neural networks, we implemented Gradient-weighted Class Activation Mapping (Grad-CAM). Grad-CAM tracks the gradients of the diagnostic output score with respect to the feature map activations of the final convolutional layer (`out_relu` in MobileNetV2). This highlights the spatial regions in the radiograph that most heavily influenced the network's prediction.

The mathematical process is structured as follows:

- **Feature Map Activation:** Extract the activation maps A^k of the final convolutional layer, and calculate the gradient of the class score y^c with respect to the activations, denoted as $\partial y^c / \partial A^k$.
- **Channel-Wise Importance:** Compute the importance weights by average-pooling the gradients over the spatial dimensions: $\alpha_k^c = (1 / Z) * \Sigma_i \Sigma_j (\partial y^c / \partial A_{i,j}^k)$.
- **Rectified Linear Combination:** Perform a weighted summation of feature maps and filter through a ReLU activation to focus only on features that positively contribute to the class: $L_{Grad\text{-}CAM} = ReLU(\Sigma_k \alpha_k^c A^k)$.
- **Heatmap Synthesis:** Bilinearly interpolate the resulting activation map to match the input scan dimensions and blend using a JET colormap overlay.

5. LLM Integration & Deterministic Fallbacks

To assist clinicians with documentation, the platform connects to the Google Gemini API (`gemini-1.5-flash`) using structured JSON prompting. The model parses the diagnostic output and confidence intervals to draft a clinical report outlining findings, suspected pathologies, and recommended follow-up actions. To guarantee high reliability in environments with strict firewalls or API rate limits, we built a deterministic local fallback report generator. If the connection fails, the backend immediately transitions to generating reports from structured medical templates, ensuring the system remains fully functional.

The system also enforces a configurable confidence threshold (default 70%). If a prediction falls below this threshold, the API flags the result with a warning label ('Low Confidence Prediction') in both the user interface and the exported PDFs, indicating that immediate clinician review is required.

6. REST API Design & Database Schema

The backend application is developed using FastAPI. FastAPI provides an asynchronous event loop that allows non-blocking operations. Heavy operations (image decoding, model inference, and Grad-CAM matrix math) are offloaded to separate background threads to prevent blocking incoming requests. The API includes automatically documented Swagger UI (`/docs`) and ReDoc (`/redoc`) endpoints.

Method	Endpoint	Description	Status Codes
POST	`/api/predict`	Validate and process upload, run inference, generate Grad-CAM and PDF, log to DB	201, 400, 503

GET	**`/api/history`**	Query SQLite prediction records (supports search and sorting)	200, 500
GET	**`/api/history/{id}`**	Fetch complete prediction and clinical report details	200, 404
DELETE	**`/api/history/{id}`**	Delete prediction record and purge images from filesystem	204, 404
GET	**`/api/health`**	System health report (DB connection and model cache status)	200

We use SQLAlchemy ORM to manage database logs. By default, the service uses an SQLite backend for easy evaluation. However, the database connection is abstracted, allowing a seamless switch to PostgreSQL by changing the database connection string environment variable.

7. Frontend UI Implementation

The client-side dashboard is built with React 19, TypeScript, and Vite. The user interface prioritizes utility and responsiveness, offering a clinical dashboard, a dedicated history view, and a workspaces area for analysis. Key interface components include:

- **Clinical Dashboard:** Renders aggregated metrics (total scans, diagnostic ratios, average confidence) fetched dynamically from the database API.
- **Workspaces:** Implements drag-and-drop file uploaders, client-side validation (file size limit and formats), and live progress indicators.
- **Interactive Grad-CAM Slider:** A double-layer image visualizer with a cross-fade slider. Clinicians can adjust the slider to smoothly transition between the raw anatomical X-ray structures and the model activation heatmaps.
- **Report & PDF Export:** Renders structured clinical summaries and supports direct client-side PDF downloads utilizing jsPDF.

8. Deployment & Containerization

For hosting, we configured a multi-stage Dockerfile and docker-compose.yml. The builder stage utilizes Node 20 to compile the React static assets, which are copied directly into the Python 3.10-slim runtime container. This design aggregates both backend routes and frontend static files under a single port (8000), making deployment clean and simple. The container is configured with health checks targeting the `/api/health` endpoint, and volumes are mapped to persist uploaded assets, databases, and application logs.

9. Limitations & Future Work

Limitations: Currently, the classifier handles binary normal-vs-pneumonia screening. In real clinical scenarios, pediatric patients often present with multiple co-occurring pathologies (e.g., pleural effusion, atelectasis, cardiomegaly).

Future Work:

- **Multi-Label Classification:** Transition the model to a multi-label classification head using sigmoid activations across 14 target classes.
- **Comparative Saliency Mapping:** Integrate secondary explainability models (like Integrated Gradients or LIME) to complement Grad-CAM.
- **PACS Connectivity:** Implement DICOM routing to allow the application to query and retrieve scans directly from clinical Picture Archiving and Communication Systems (PACS).

10. Conclusion & Legal Disclaimer

The MedTool AI Medical Intelligence Platform demonstrates a complete, production-ready full-stack software system. By packaging complex ML pipelines, explainability models, and generative AI reporting inside a robust web interface, the application bridges the gap between deep learning research and clinical workflow support.

Legal Disclaimer: This application is built for educational, research, and technical evaluation purposes only. It is not approved for diagnostic clinical use. All diagnostic assessments must be verified by a licensed clinical radiologist.